CHAPTER 22

Less-Numerical Algorithms

22.0 Introduction

You can stop reading now. You are done with *Numerical Recipes*, as such. This final chapter is an idiosyncratic set of "*less*-numerical recipes" that, for one reason or another, we have decided to include between the covers of an otherwise *more*-numerically oriented book. Authors of computer science texts, we've noticed, like to throw in a token numerical subject (usually quite a dull one — quadrature, for example). We find that we are not free of the reverse tendency.

Our selection of material is not completely arbitrary. In §9.0 we promised to provide a simple plotting routine. Another promised topic, Gray codes, was already used in the construction of quasi-random sequences (§7.8) and here needs only some additional explication. Two other topics, on diagnosing a computer's floating-point parameters, and on arbitrary precision arithmetic, give additional insight into the machinery behind the casual assumption that computers are useful for doing things with real numbers (as opposed to integers or characters). The latter of these topics also shows a very different use for Chapter 12's fast Fourier transform.

The three other topics (checksums, Huffman, and arithmetic coding) involve different aspects of data coding, compression, and validation. The material here is intended to be somewhat less abstract, and somewhat more practical, than the discussion of coding in §16.2, where coding was used to illustrate statistical aspects of state estimation. If you handle a large amount of data (numerical data, even), then a passing familiarity with these subjects might at some point come in handy. In §13.6, for example, we already encountered a good use for Huffman coding.

But again, you don't have to read this chapter. (And you should learn about quadrature from Chapters 4 and 17, not from a computer science textbook!)

22.1 Plotting Simple Graphs

Yes, we all have our favorite plotting or graphics packages, and our favorite ways of generating plots from within C++ programs. But wait: Are your C++ pro-

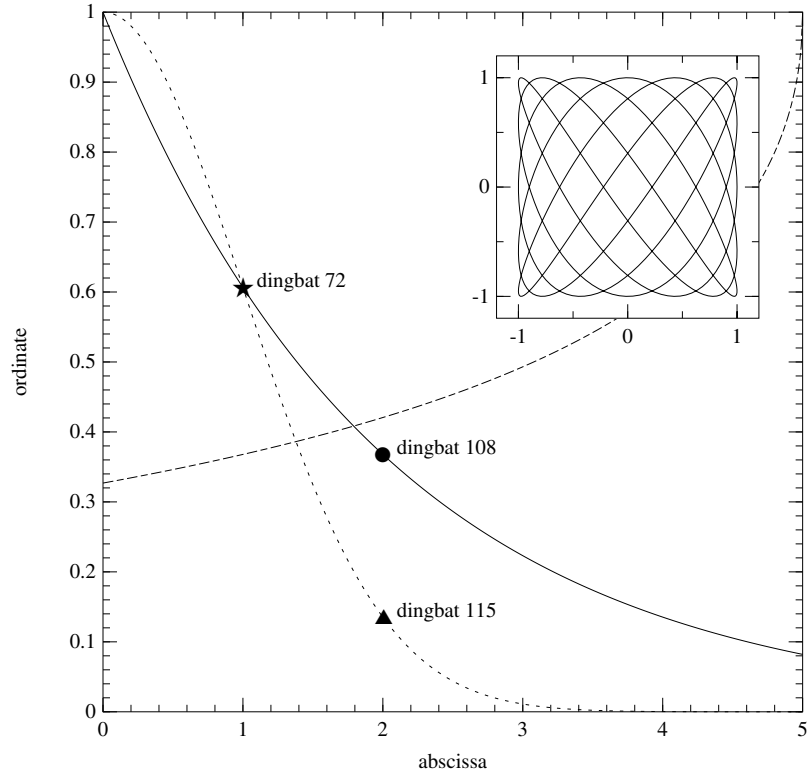


Figure 22.1.1. Simple plot generated using the objects `PSpage` and `PSplot`, which are wrappers for generating PostScript.

grams generating long text files of numbers, just so that you can read those numbers into a separate plotting or graphics package? If so, you might benefit from this section.

We find it useful to have on hand a couple of short C++ objects, implemented in simple source code, that generate simple plots. (By "simple," we mean "like most of the figures in this book.") We are then able to make plots from anywhere in our programs, whether as an aid in debugging or as final output. Equally important, we can make changes to the plotting source code at will, adding features or modifying the look of the plot.

One way of accomplishing these goals is by means of a C++ "wrapper" that does no more, nor less, than to write out a valid PostScript [1] file, which can be viewed or printed using a PostScript viewer such as the freely available Ghostscript/GSview [2]. In fact, the viewer can readily be invoked by a method within the wrapper object, so that the plot simply pops up in its own window on your screen.

An example will make this clearer. Figure 22.1.1 shows a sample plot that has a couple of x, y scaled boxes, some lines and points of varying types, and some text labels. Here is the code that generates the figure:

psplotexample.h

```
void psplot_example() {
Routine for creating Figure 22.1.1.
    VecDoub x1(500),x2(500),y1(500),y2(500),y3(500),y4(500);
    for (Int i=0;i<500;i++) {                    Generate some data.
```

```
        x1[i] = 5.*i/499.;
        y1[i] = exp(-0.5*x1[i]);
        y2[i] = exp(-0.5*SQR(x1[i]));
        y3[i] = exp(-0.5*sqrt(5.-x1[i]));
        x2[i] = cos(0.062957*i);
        y4[i] = sin(0.088141*i);
    }

    PSpage pg("d:\\nr3\\newchap20\\myplot.ps");   Instantiate a page.
    PSplot plot1(pg,100.,500.,100.,500.);        Instantiate a plot on the page. Position is specified in pt (72 pt = 1 in, or 28 pt = 1 cm).
    plot1.setlimits(0.,5.,0.,1.);
    plot1.frame();
    plot1.autoscales();
    plot1.xlabel("abscissa");
    plot1.ylabel("ordinate");
    plot1.lineplot(x1,y1);
    plot1.setdash("2 4");
    plot1.lineplot(x1,y2);
    plot1.setdash("6 2 4 2");
    plot1.lineplot(x1,y3);
    plot1.setdash("");                           Unsets dash.
    plot1.pointsymbol(1.,exp(-0.5),72,16.);
    plot1.pointsymbol(2.,exp(-1.),108,12.);
    plot1.pointsymbol(2.,exp(-2.),115,12.);
    plot1.label("dingbat 72",1.1,exp(-0.5));
    plot1.label("dingbat 108",2.1,exp(-1.));
    plot1.label("dingbat 115",2.1,exp(-2.));

    PSplot plot2(pg,325.,475.,325.,475.);        Instantiate a second plot.
    plot2.clear();                               Erase what's underneath it.
    plot2.setlimits(-1.2,1.2,-1.2,1.2);
    plot2.frame();
    plot2.scales(1.,0.5,1.,0.5);
    plot2.lineplot(x2,y4);

    pg.close();
    pg.display();                                Pop up a window displaying the plot file.
}
```

The general idea is that a `PSpage` object (`pg` in the example above) represents a whole sheet of paper, or window on the screen. It can contain one or more `PSplot` objects. In the above example there are two: `plot1` and `plot2`. `PSplot` objects can be separate on the page, or overlapping. Each has its own x, y coordinate system, its own x- and y-axis labels, and so forth. With no more explanation than this, you should be able to find a program line above that corresponds to each feature in the figure. The last line makes the plot pop up on our screen.

Point symbols are referenced by their character number in the Zapf Dingbats font, which is built into PostScript. If you want to see all the possible symbols, a Web search for "LaTeX Postscript Dingbats" will turn up several charts. Or, just write a program to plot them all. (Hint: There are possibly useful symbols from 33 to 126, and from 161 to 254.)

A Webnote [3] gives the complete source code for the `PSpage` and `PSplot` objects, which is only about 150 lines long. In the course of writing this book, our personal version of the code expanded to about 450 lines. This is an order of magnitude or two less than the standard packages that are available in open source code, GNUPLOT, for example [4]. It is a question of trading off capability (theirs much greater) for ease of modifying the source code (you be the judge).

If you choose to go down this road, you'll soon want to learn more of PostScript as a language. A good reference is [5].

CITED REFERENCES AND FURTHER READING:

Adobe Systems, Inc. 1999, *PostScript Language Reference*, 3rd ed. (Reading, MA: Addison-Wesley).[1]

Ghostscript and GSview 2007+, at `http://www.cs.wisc.edu/~ghost/`.[2]

Numerical Recipes Software 2007, "Code for PSpage and PSplot," *Numerical Recipes Webnote No. 26*, at `http://www.nr.com/webnotes?26` [3]

GNUPLOT 2007+, at `http://www.gnuplot.info`.[4]

McGilton, H., and Campione, M. 1992, *PostScript by Example* (Reading, MA: Addison-Wesley).[5]

22.2 Diagnosing Machine Parameters

A convenient fiction is that a computer's floating-point arithmetic is "accurate enough." If you believe this fiction, then numerical analysis becomes a very clean subject. Roundoff error disappears, and many finite algorithms are exact. Only manageable truncation error (§1.1) stands between you and a perfect calculation. Sounds rather naive, doesn't it?

Yes, it is naive. Notwithstanding, we have adopted this fiction throughout most of this book. To do a good job of answering the question of how roundoff error propagates, or can be bounded, for every algorithm that we have discussed would be impractical. In fact, it would not be possible: Rigorous analysis of many practical algorithms has never been made, by us or anyone.

Almost all processors today share the same floating-point data representation, namely that specified in IEEE Standard 754-1985 [1], and therefore the same strengths and weaknesses as regards roundoff error. But this was not always so! The history of computing is full of machines with strange floating-point representations by modern standards. Many early computers had 36-bit words, typically partitioned as a sign bit, 8 bits of exponent, and 27 bits of mantissa. The influential IBM 7090/7094 series was of this type. The legendary CDC 6600 and 7600 machines, designed by Seymour Cray, had 60-bit words (sign, 11-bit exponent, 48-bit mantissa). A particularly odd design was the IBM STRETCH, whose 64 bits were allocated to an exponent flag bit, 10 exponent bits, the exponent sign, a 48-bit mantissa, its sign, and three flag bits. The exponent flag bit was used to signal overflow or underflow, while the other flag bits could be set by the user to indicate — anything! So let us all be grateful for IEEE 754.

Likewise, almost all numerical computing today is done in double precision, that is, in 64-bit words, what C++ defines as `double` and we denote as `Doub`. This, also, was not always so. It has happened (one might argue) because the availability of memory has increased even more rapidly than the appetite for it in numerical computation. Many programmers born before 1960 still feel a small frisson when they type `double` instead of `float`. Indeed, the vast majority of routines in this book will work just fine, for the vast majority of applications, with merely `float` precision. In most cases, the use of `double` simply serves to reinforce an erroneous belief in the above "convenient fiction."

Still, every once in a while, you will need to know what the limitations of your machine's floating-point arithmetic actually are — the more so when your treatment of floating-point roundoff error is going to be intuitive, experimental, or casual. This will certainly be true if you ever encounter a processor with nonstandard (that is, non-IEEE compliant) hardware. Such processors still do exist, though generally hidden away in embedded special-purpose devices.

If you are lucky, then calls to the methods in the C++ standard library class `numeric_limits` will tell you what you need to know. It is a good idea to familiarize yourself with that class, including some of its esoterica, like `round_style` and `has_denorm` [2].

A more experimental approach is to use methods that were developed to ferret out machine parameters in the bad old days before standards [3,4], especially parameters that were supposed to be transparent to the (ordinary) user. The object `Machar`, listed in full in a Webnote [5], gives an implementation of a number of of these methods. The quantities determined are

- `ibeta` is the radix in which numbers are represented, almost always 2, but historically sometimes 16, or even 10.
- `it` is the number of base-`ibeta` digits in the floating-point mantissa M.
- `machep` is the exponent of the smallest (most negative) power of `ibeta` that, added to 1.0, gives something different from 1.0.
- `eps` is the floating-point number $\texttt{ibeta}^{\texttt{machep}}$, loosely referred to as the "floating-point precision."
- `negep` is the exponent of the smallest power of `ibeta` that, subtracted from 1.0, gives something different from 1.0.
- `epsneg` is $\texttt{ibeta}^{\texttt{negep}}$, another way of defining floating-point precision. Not infrequently, `epsneg` is 0.5 times `eps`; occasionally `eps` and `epsneg` are equal.
- `iexp` is the number of bits in the exponent (including its sign or bias).
- `minexp` is the smallest (most negative) power of `ibeta` consistent with there being no leading zeros in the mantissa.
- `xmin` is the floating-point number $\texttt{ibeta}^{\texttt{minexp}}$, generally the smallest (in magnitude) useable floating value.
- `maxexp` is the smallest (positive) power of `ibeta` that causes overflow.
- `xmax` is $(1 - \texttt{epsneg}) \times \texttt{ibeta}^{\texttt{maxexp}}$, generally the largest (in magnitude) useable floating value.
- `irnd` returns a code in the range $0 \ldots 5$, giving information on what kind of rounding is done in addition, and on how underflow is handled. See below.
- `ngrd` is the number of "guard digits" used when truncating the product of two mantissas to fit the representation.

The parameter `irnd` needs some additional explanation. In the IEEE standard, bit patterns correspond to exact, "representable" numbers. The specified method for rounding an addition is to add two representable numbers "exactly," and then round the sum to the closest representable number. If the sum is precisely halfway between two representable numbers, it should be rounded to the even one (low-order bit zero). The same behavior should hold for all the other arithmetic operations, that is, they should be done in a manner equivalent to infinite precision, and then rounded to the closest representable number.

If `irnd` returns 2 or 5, then your processor is compliant with this standard. If it

Sample Results Returned by `Machar`			
	IEEE-compliant processor		historical
precision	`float`	`double`	DEC-VAX
`ibeta`	2	2	2
`it`	24	53	24
`machep`	−23	−52	−24
`eps`	1.19×10^{-7}	2.22×10^{-16}	5.96×10^{-8}
`negep`	−24	−53	−24
`epsneg`	5.96×10^{-8}	1.11×10^{-16}	5.96×10^{-8}
`iexp`	8	11	8
`minexp`	−126	−1022	−128
`xmin`	1.18×10^{-38}	2.23×10^{-308}	2.94×10^{-39}
`maxexp`	128	1024	127
`xmax`	3.40×10^{38}	1.79×10^{308}	1.70×10^{38}
`irnd`	5	5	1
`ngrd`	0	0	0

returns 1 or 4, then it is doing some kind of rounding, but not the IEEE standard. If `irnd` returns 0 or 3, then it is truncating the result, not rounding it — not desirable.

The other issue addressed by `irnd` concerns underflow. If a floating value is less than `xmin`, many computers underflow its value to zero. Values `irnd` $= 0, 1,$ or 2 indicate this behavior. The IEEE standard specifies a more graceful kind of underflow: As a value becomes smaller than `xmin`, its exponent is frozen at the smallest allowed value while its mantissa is decreased, acquiring leading zeros and "gracefully" losing precision. This is indicated by `irnd` $= 3, 4,$ or 5.

Sometimes results can be compiler-dependent. For example, some compilers underflow intermediate results ungracefully, yielding `irnd` $= 2$ rather than 5.

Call the `report` method in `Machar` to see the comparison between its results and those returned by `numeric_limits`. Some values returned by `Machar` for IEEE compliant processors are given in the table above and compared with an important historical processor, the DEC-VAX. This processor, like its predecessor PDP-11, used a representation with a "phantom" leading 1 bit in the mantissa. You can see that this achieved a smaller `eps` for the same wordlength but could not underflow gracefully, since there were no denormalized numbers.

CITED REFERENCES AND FURTHER READING:

IEEE Standard for Binary Floating-Point Numbers, ANSI/IEEE Std 754–1985 (New York: IEEE, 1985).[1]

Josuttis, N.M. 1999, *The C++ Standard Library: A Tutorial and Reference* (Boston: Addison-Wesley), §4.3.[2]

Cody, W.J. 1988, "MACHAR: A Subroutine to Dynamically Determine Machine Parameters," *ACM Transactions on Mathematical Software*, vol. 14, pp. 303–311.[3]

Malcolm, M.A. 1972, "Algorithms to Reveal Properties of Floating-Point Arithmetic," *Communications of the ACM*, vol. 15, pp. 949–951.[4]

Numerical Recipes Software 2007, "Code for Machar," *Numerical Recipes Webnote No. 27*, at `http://www.nr.com/webnotes?27` [5]

Goldberg, D. 1991, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, vol. 23, pp. 5–48.

22.3 Gray Codes

A Gray code is a function $G(i)$ of the integers i that for each integer $N \geq 0$ is one-to-one for $0 \leq i \leq 2^N - 1$, and that has the following remarkable property: The binary representations of $G(i)$ and $G(i+1)$ differ in *exactly one bit*. An example of a Gray code (in fact, the most commonly used one) is the sequence 0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100, 1100, 1101, 1111, 1110, 1010, 1011, 1001, and 1000, for $i = 0, \ldots, 15$. The algorithm for generating this code is simply to form the bitwise exclusive-or (XOR) of i with $i/2$ (integer part). Think about how the carries work when you add one to a number in binary, and you will be able to see why this works. You will also see that $G(i)$ and $G(i+1)$ differ in the bit position of the rightmost zero bit of i (prefixing a leading zero if necessary).

The spelling is "Gray," not "gray": The codes are named after one Frank Gray, who first patented the idea for use in shaft encoders. A shaft encoder is a wheel with concentric coded stripes, each of which is "read" by a fixed optical sensor or conducting brush. The idea is to generate a binary code describing the angle of the wheel. The obvious, but wrong, way to build a shaft encoder is to have one stripe (the innermost, say) present on half the wheel, but absent on the other half; the next stripe is present in quadrants 1 and 3; the next stripe is present in octants 1, 3, 5, and 7; and so on. The optical or electrical sensors together then read a direct binary code for the position of the wheel.

The reason this method is bad is that there is no way to guarantee that all the brushes will make or break contact *exactly* simultaneously as the wheel turns. Going from position 7 (0111) to 8 (1000), one might pass spuriously and transiently through 6 (0110), 14 (1110), and 10 (1010), as the different brushes make or break contact. Use of a Gray code on the encoding stripes guarantees that there is no transient state between 7 (0100 in the sequence above) and 8 (1100).

Of course we then need circuitry, or algorithmics, to translate from $G(i)$ to i. Figure 22.3.1(b) shows how this is done by a cascade of XOR gates. The idea is that each output bit should be the XOR of all more significant input bits. To do N bits of Gray code inversion requires $N - 1$ steps (or gate delays) in the circuit. (Nevertheless, this is typically very fast in circuitry.) In a register with word-wide binary operations, we don't have to do N consecutive operations, but only $\ln_2 N$. The trick is to use the associativity of XOR and group the operations hierarchically. This involves sequential right-shifts by $1, 2, 4, 8, \ldots$ bits until the wordlength is exhausted. Here is a piece of code for doing both $G(i)$ and its inverse:

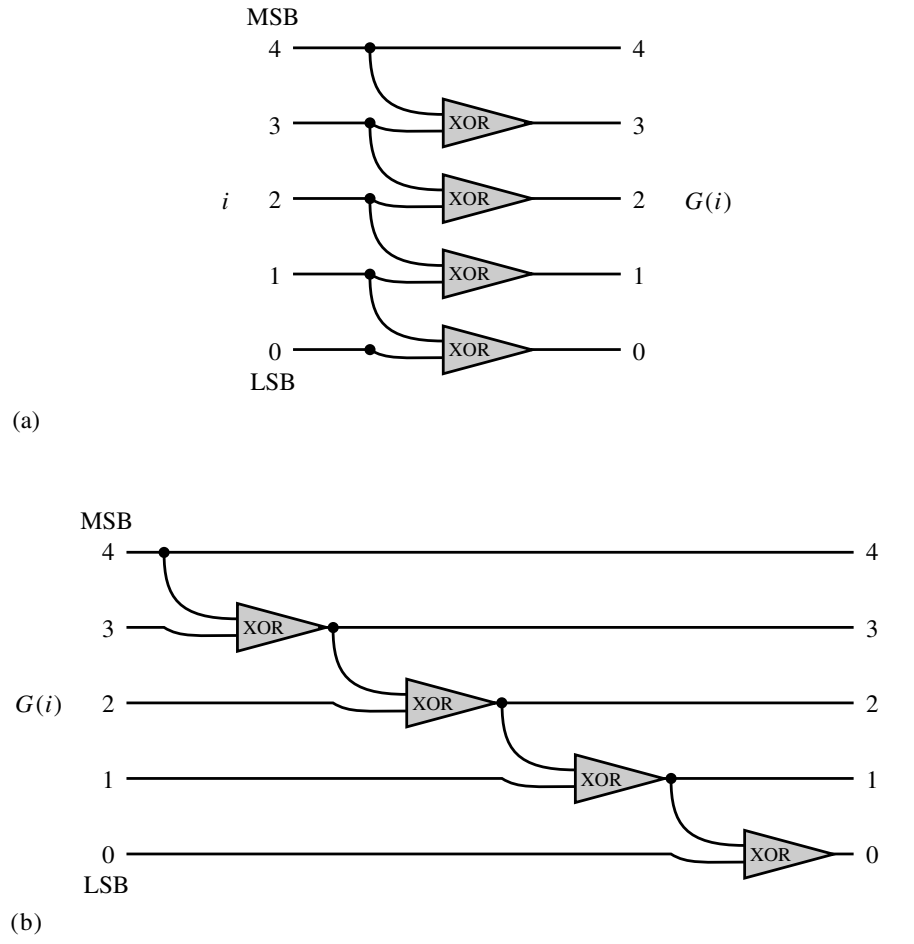


Figure 22.3.1. Single-bit operations for calculating the Gray code $G(i)$ from i (a), or the inverse (b). LSB and MSB indicate the least and most significant bits, respectively. XOR denotes exclusive-or.

```
struct Gray {                                                    igray.h
```
Methods for the Gray code and its inverse.

```
    Uint gray(const Uint n) {return n ^ (n >> 1);}
```
Return the Gray code of an integer `n`. This is the easy direction!

```
    Uint invgray(const Uint n) {
```
Return the inverse of the Gray code.

```
        Int ish=1;
        Uint ans=n,idiv;
        for (;;) {                  In hierarchical stages, starting with a one-bit right-shift,
            ans ^= (idiv=ans >> ish);   cause each bit to be XORed with all more sig-
            if (idiv <= 1 || ish == 16) return ans;   nificant bits.
            ish <<= 1;              Double the amount of shift on the next cycle.
        }
    }
};
```

In numerical work, Gray codes can be useful when you need to do some task

that depends intimately on the bits of i, looping over many values of i. Then, if there are economies in repeating the task for values differing by only one bit, it makes sense to do things in Gray code order rather than consecutive order. We saw an example of this in §7.8, for the generation of quasi-random sequences.

CITED REFERENCES AND FURTHER READING:

Horowitz, P., and Hill, W. 1989, *The Art of Electronics*, 2nd ed. (New York: Cambridge University Press), §8.02.

Knuth, D.E. 2005, *Generating All Tuples and Permutations*, fascicle 2 of vol. 4 of *The Art of Computer Programming* (Upper Saddle River, NJ: Addison-Wesley), §7.2.1.1.

22.4 Cyclic Redundancy and Other Checksums

There are networks all around you: not just "the" Internet with its IP and TCP protocols, but also *embedded* networks that move bits around within a device or among closely coupled devices. Examples include the *SMBus* network that communicates power management information between smart batteries and the devices that they power, or the *Bluetooth* network that connects cell phones to nearby accessories. We wouldn't be overly surprised to find a network inside of our wristwatch or electric toothbrush!

Different networks have different protocols, but standard engineering practice is to package the raw information into packets with fixed or variable numbers of bits. Packet lengths are typically in the range from a few tens to a few thousand bits. Smaller would imply proportionally too much overhead per packet, while longer would make excessive demands on buffer sizes, collision avoidance, etc.

When a packet is sent from point A to point B, one wants to know that it has arrived without error. The simplest form of insurance is to add a "parity bit," chosen so as to make the total number of one-bits (versus zero-bits) either always even ("even parity") or always odd ("odd parity"). Any *single-bit* error in a packet will thereby be detected. When errors are sufficiently rare, or their consequence sufficiently minor, use of parity provides enough error detection. For example, the ASCII character set was originally designed for 7-bit characters, with an 8th parity bit.

Since the parity bit has two possible values (0 and 1), it has, on average, only a 50% chance of detecting an erroneous packet with *multiple* wrong bits. That is not nearly good enough for most applications. Most communications protocols [1] use a multibit generalization of the parity bit called a "cyclic redundancy check" or CRC. Often, the CRC is 16 bits (two bytes) long. Then the chance of a random set of errors going undetected is 1 in $2^{16} = 65536$.

Now enters mathematics. It is easy to find M-bit CRCs that have the property of detecting *all* errors that occur in M or fewer *consecutive* bits, for any length of message. (We prove this below.) Since noise in communication channels tends to be "bursty," with short sequences of adjacent bits getting corrupted, this consecutive-bit property is highly desirable. Furthermore, for packets with a fixed (or bounded) payload size of N bits, one can find CRCs that find all occurrences of D or fewer errors *anywhere* in the payload. Obviously, the game is to find the CRC that maximizes D. The value $D + 1$ is the *Hamming distance* of the CRC for that value of N using that checksum (cf. §16.2).

Useful 16-bit CRC Polynomials (after [3])			
j	Name	Polynomial	Best N (bits)
0		`0x755B` $= (x^3+x^2+1)^*(x^6+x^5+x^2+x+1)^*(x^7+x^3+1)^*$	242–2048+
1		`0xA7D3` $= (x^3+x^2+1)^*(x^6+x^5+x^2+x+1)^*(x^7+x^6+x^5+x^4+1)^*$	256–2048+
2	ANSI-16	`0x8005` $= (x+1)(x^{15}+x+1)^*$	242–2048+
3	CCITT-16	`0x1021` $= (x+1)(x^{15}+x^{14}+x^{13}+x^{12}+x^4+x^3+x^2+x+1)^*$	242–2048+
4		`0x5935` $= (x^{16}+x^{14}+x^{12}+x^{11}+x^8+x^5+x^4+x^2+1)$	136–241
5		`0x90D9` $= (x+1)(x^{15}+x^{11}+x^{10}+x^9+x^8+x^7+x^5+x^4+x^2+x+1)$	20–135
6	IEC-16	`0x5B93` $= (x+1)(x+1)(x^7+x^6+x^3+x+1)^*(x^7+x^6+x^5+x^4+x^3+x^2+1)^*$	20–112
7		`0x2D17` $= (x^2+x+1)^*(x^{14}+x^{13}+x^9+x^7+x^5+x^4+1)$	16–19
* denotes primitive factor			

The design of CRCs lies in the province of communications software experts and chip-level hardware designers — people with bits under their fingernails. A passing familiarity with some of the concepts involved can be useful, however, both because the mathematics involved has connections to other applications (for example, random number generation, cf. §7.1 and §7.5), and because you might actually want to add a couple of bytes of checksum to your own data records in some applications where you are handling, or moving, large amounts of data.

Sometimes CRCs can be used to compress data as they are recorded. If identical data records occur frequently, one can keep sorted in memory the CRCs of previously encountered records. A new record is archived in full if its CRC is different, otherwise only a pointer to a previous record need be archived. In this application one might use 8 bytes of CRC, to make the odds of mistakenly discarding a different data record tolerably small; or, if previous records can be randomly accessed, a full comparison can be made to decide whether records with identical CRCs are in fact identical.

Now let us briefly discuss the theory of CRCs. After that, we will give an implementation that generates 16-bit CRCs that are known to be particularly good, or else are enshrined as standard (and, it turns out, this is not the same thing!).

The mathematics underlying CRCs is "polynomials over the integers modulo 2." Any binary message can be thought of as a polynomial with coefficients 0 and 1. For example, the message "1100001101" is the polynomial $x^9 + x^8 + x^3 + x^2 + 1$. Since 0 and 1 are the only integers modulo 2, a power of x in the polynomial is either present (1) or absent (0).

An M-bit-long CRC is based on a polynomial of degree M, called the generator polynomial. Given the generator polynomial G (which can be written either in polynomial form or as a bit-string, e.g., 10001000000100001 for $x^{16} + x^{12} + x^5 + 1$), here is how you compute the CRC for a sequence of bits S: First, multiply S by x^M, that is, append M zero bits to it. Second, divide, by long division, G into Sx^M. Keep in mind that the subtractions in the long division are done modulo 2, so that there are never any "borrows": Modulo 2 subtraction is the same as logical exclusive-or (XOR). Third, ignore the quotient you get. Fourth, when you eventually get to a remainder, it is the CRC; call it C. C will be a polynomial of degree $M - 1$ or less, otherwise you would not have finished the long division. Therefore, in bit-string form, it has M bits, which may include leading zeros. (C might even be all zeros; see below.)

If you work through the above steps in an example, you will see that most of what you write down in the long-division tableau is superfluous. You are actually just left-shifting sequential bits of S, from the right, into an M-bit register. Every time a 1 bit gets shifted off the left end of this register, you zap the register by an XOR with the M low-order bits of G (that is, all the bits of G except its leading 1). When a 0 bit is shifted off the left end you don't zap the register. When the last bit that was originally part of S gets shifted off the left end of the register, what remains is the CRC.

You can immediately recognize how efficiently this procedure can be implemented in hardware. It requires only a shift register with a few hard-wired XOR taps into it. That is how CRCs are computed in communications devices, taking a tiny part of a single chip. In software, the implementation is not so elegant, since bit-shifting is not generally very efficient. One therefore typically finds (as in our implementation below) table-driven routines that pre-calculate the result of a bunch of shifts and XORs, say for each of 256 possible 8-bit inputs [2].

Every generator polynomial of degree M with a nonzero x^0 term yields a CRC that detects *all* possible combinations of errors in any *frame* of M consecutive bits. (A special case of this is that it detects any single-bit error in a message of arbitrary length N.) To see how this works, suppose two messages, S and T, differ only within a frame of M bits. Then their CRCs differ by an amount that is the remainder when G is divided into $(S - T)x^M \equiv R$. Now R has the form of leading zeros (which can be ignored), followed by some 1's in an M-bit frame, followed by trailing zeros (which are just multiplicative factors of x): $R = x^n F$, where F is a polynomial of degree at most $M - 1$ and $n > 0$. But since G has a nonzero x^0 term, it is not divisible by x. So G cannot divide R. Therefore S and T must have different CRCs.

What about two-bit errors, not necessarily in a frame of size M? That leads us to *primitive polynomials*: A polynomial over the integers modulo 2 may be irreducible, meaning that it can't be factored. A subset of the irreducible polynomials is the primitive polynomials. These generate maximum length sequences when used in shift registers, as described in §7.5. The polynomial $x^2 + 1$ is not irreducible: $x^2 + 1 = (x + 1)(x + 1)$, so it is also not primitive. The polynomial $x^4 + x^3 + x^2 + x + 1$ is irreducible, but it turns out not to be primitive. The polynomial $x^4 + x + 1$ is both irreducible and primitive.

Primitive polynomials are here interesting because they have a very high order. Don't confuse *order* with *degree*: The order e of a polynomial is the smallest integer e such that the polynomial divides (in the present mod 2 case) $x^e + 1$. Primitive

polynomials, it turns out, have the largest possible order e for their degree n, given by

$$e = 2^n - 1 \tag{22.4.1}$$

(In fact, this is why their shift registers have maximum length.) If two messages differ on exactly two bits, spaced k bits apart, then their difference is $x^k + 1$ times some trailing powers of x. If the generator G contains a primitive factor of order e, then G can't possibly divide this difference, as long as $k < e$.

Thus, a primitive factor of degree n guarantees two-bit error detection for spacings up to $2^n - 1$. For this reason, generators are often chosen to be primitive polynomials of degree M. Alternatively, the generator may be chosen be a primitive polynomial times $(1 + x)$, which turns out to detect parity errors for all message sizes N, while the range of two-bit detections is reduced only by a factor of 2.

A number of "standard" CRC polynomials were chosen by no other criteria, sometimes adding only the criterion that they should have only a small number of terms. (This was at one time important for hardware design.) For example, the CCITT (Comité Consultatif International Télégraphique et Téléphonique) has anointed $x^{16} + x^{12} + x^5 + 1$ as "CCITT-16"; it is the product of $x + 1$ and a primitive polynomial. The polynomial ANSI-16 (see table on p. 1169) also has this character.

Similarly for some choices other than 16 bits: "CRC-12" is $(x + 1)(x^{11} + x^2 + 1)$, the latter factor being primitive. The most common 32-bit CRC, "CRC-32," used in the ethernet standard (IEEE 802.3) and elsewhere, is $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$, which is primitive.

Now here is something relatively new in this ancient field [3]: For carefully chosen generators G, all two-bit errors in a packet with payload size N can be detected *even if* $e < N$. This is because the previous argument was sufficient, but not necessary: A cleverly chosen G can fail to divide $x^k - 1$ for other reasons than having a primitive factor of large order. This idea opens up the design space to search, essentially by brute force, for generators that have $D > 2$, that is, are capable of finding not just all two-bit errors, but all three-bit errors, all four-bit errors, etc., up to some bound that depends on N and M. Several of these "new" generators are shown in the table on p. 1169, which is based on [3] (which see for details), along with their recommended values of N. A generator that is good for large N is not necessarily good for small N, and vice versa, so you should stick to the recommended values. The hexadecimal values in the table give binary representations of the polynomials, with the convention that each must be prefaced by a leading 1 (the x^{16} term).

In most protocols, a transmitted block of data consists of some N data bits, directly followed by the M bits of their CRC (or the CRC XORed with a constant; see below). There are two equivalent ways of validating a block at the receiving end. Most obviously, the receiver can compute the CRC of the data bits, and compare it to the transmitted CRC bits. Less obviously, but more elegantly, the receiver can simply compute the CRC of the total block, with $N + M$ bits, and verify that a result of zero is obtained. Proof: The total block is the polynomial $Sx^M + C$ (data left-shifted to make room for the CRC bits). The definition of C is that $Sx^m = QG + C$, where Q is the discarded quotient. But then $Sx^M + C = QG + C + C = QG$ (remember modulo 2), which is a perfect multiple of G. It remains a multiple of G when it gets multiplied by an additional x^M on the receiving end, so it has a zero CRC, q.e.d.

A couple of small variations on the basic procedure need to be mentioned [1]: First, when the CRC is computed, the M-bit register need not be initialized to zero.

Initializing it to some other M-bit value (e.g., all 1's) in effect prefaces all blocks by a phantom message that would have given the initialization value as its remainder. It is advantageous to do this, since the CRC described thus far otherwise cannot detect the addition or removal of any number of initial zero bits. (Loss of an initial bit, or insertion of zero bits, are common "clocking errors.") Second, one can add (XOR) any M-bit constant K to the CRC before it is transmitted. This constant can either be XORed away at the receiving end, or else it just changes the expected CRC of the whole block by a known amount, namely the remainder of dividing G into Kx^M. The constant K is frequently "all bits," changing the CRC into its ones complement. This has the advantage of detecting another kind of error that the CRC would otherwise not find: deletion of an initial 1 bit in the message with spurious insertion of a 1 bit at the end of the block.

The following object `Icrc` implements the calculation of 16-bit CRCs for the generators listed in the table. The constructor sets which generator is to be used, and also whether the initial register should be all bits (the default) or zero. `Icrc` is loosely based on the function in [2]. Here is how to understand its operation: First look at the function `icrc1`. This is used only by the constructor, to initialize a table of length 256, incorporating one character into a 16-bit CRC register. The only trick used is that a character's bits are XORed into the most significant bits of the register, all eight together, instead of being fed into the least significant bit, one bit at a time, at the time of the register shift. This works because XOR is associative and commutative — we can feed in character bits *any* time before they will determine whether to zap with the generator polynomial.

Now look at the methods `crc` and `concat`. Go back to thinking about a character's bits being shifted into the CRC register from the least significant end. The key observation is that while 8 bits are being shifted into the register's low end, all the generator zapping is being determined by the bits already in the high end. Since XOR is commutative and associative, all we need is a table of the results of all this zapping, for each of 256 possible high-bit configurations. Then we can play catch-up and XOR an input character into the result of a lookup into this table. But this is exactly the table that was constructed by `icrc1`. References [2,4,5] give further details on table-driven CRC computations.

icrc.h

```
struct Icrc {
```

Object for computing 16-bit cyclic redundancy checksums.

```
    Uint jcrc,jfill,poly;
    static Uint icrctb[256];

    Icrc(const Int jpoly, const Bool fill=true) : jfill(fill ? 255 : 0) {
```

Constructor. Choose one of 8 generators (see table) by the value of `jpoly`. Initialize the CRC register to all bits if `fill` is true, otherwise to zero.

```
        Int j;
        Uint okpolys[8] = {0x755B,0xA7D3,0x8005,0x1021,0x5935,0x90D9,0x5B93,0x2D17};
```

Generator polynomials, see table.

```
        poly = okpolys[jpoly & 7];
        for (j=0;j<256;j++) {
            icrctb[j]=icrc1(j << 8,0);
```

Table of CRCs of all characters.

```
        }
        jcrc = (jfill | (jfill << 8));
    }
```

```
    Uint crc(const string &bufptr) {
    Initialize the CRC register, compute and return the 16-bit CRC for the string bufptr.
        jcrc = (jfill | (jfill << 8));
        return concat(bufptr);
    }

    Uint concat(const string &bufptr) {
    Without reinitializing the CRC register, compute and return the 16-bit CRC for the string
    bufptr. In effect, this appends bufptr to previous strings since the last call of crc and
    returns the overall CRC.
        Uint j,cword=jcrc,len=bufptr.size();
        for (j=0;j<len;j++) {       Loop over the characters in the string.
            cword=icrctb[Uchar(bufptr[j]) ^ hibyte(cword)] ^ (lobyte(cword) << 8);
        }
        return jcrc = cword;
    }

    Uint icrc1(const Uint jcrc, const Uchar onech) {
    Given a remainder up to now, return the new CRC after one character is added. Used by
    Icrc to initialize its table.
        Int i;
        Uint ans=(jcrc ^ onech << 8);
        for (i=0;i<8;i++) {             Here is where 8 one-bit shifts, and some XORs
            if (ans & 0x8000) ans = (ans <<= 1) ^ poly;      with the generator poly-
            else ans <<= 1;                                  nomial, are done.
            ans &= 0xffff;
        }
        return ans;
    }

    inline Uchar lobyte(const unsigned short x) {
        return (Uchar)(x & 0xff); }
    inline Uchar hibyte(const unsigned short x) {
        return (Uchar)((x >> 8) & 0xff); }
};
Uint Icrc::icrctb[256];
```

What if you need more than 16 bits of checksum? For a true 32-bit CRC, you will need to rewrite the routines given to work with a longer generating polynomial. For example, $x^{32} + x^7 + x^5 + x^3 + x^2 + x + 1$ is primitive modulo 2 and has nonleading, nonzero bits only in its least significant byte (which makes for some simplification). The idea of table lookup on only the most significant byte of the CRC register goes through unchanged.

Easier, if you don't care about the M-consecutive bit property of the checksum, is to just instantiate more than one copy of `Icrc`, each with a different generator (first argument in constructor). These provide statistically independent checks.

22.4.1 Other Kinds of Checksums

Quite different from CRCs are the various techniques used to append a decimal "check digit" to numbers that are handled by human beings (e.g., typed into a computer). Check digits need to be proof against the kinds of highly structured errors that humans tend to make, such as transposing consecutive digits. Wagner and Putter [6] give an interesting introduction to this subject, including specific algorithms.

Checksums now in widespread use vary from fair to poor. The 10-digit ISBN (International Standard Book Number) that you find on most books, including this

one, uses the check equation

$$10d_1 + 9d_2 + 8d_3 + \cdots + 2d_9 + d_{10} = 0 \pmod{11} \tag{22.4.2}$$

where d_{10} is the right-hand check digit. The character "X" is used to represent a check digit value of 10. Another popular scheme is the so-called "IBM check," often used for account numbers (including, e.g., MasterCard). Here, the check equation is

$$2\#d_1 + d_2 + 2\#d_3 + d_4 + \cdots = 0 \pmod{10} \tag{22.4.3}$$

where $2\#d$ means, "multiply d by two and add the resulting decimal digits." United States banks code checks with a nine-digit processing number whose check equation is

$$3a_1 + 7a_2 + a_3 + 3a_4 + 7a_5 + a_6 + 3a_7 + 7a_8 + a_9 = 0 \pmod{10} \tag{22.4.4}$$

The familiar 12-digit Universal Product Code (UPC) is printed with both a decimal representation and a synonymous bar code. The digits are divided into a one-digit "category," a five-digit manufacturer, a five-digit product identification, and one-digit checksum. The check equation is

$$3a_1 + a_2 + 3a_3 + a_4 + 3a_5 + \cdots + 3a_{11} + a_{12} = 0 \pmod{10} \tag{22.4.5}$$

The bar code put on many envelopes by the U.S. Postal Service is decoded by removing the single tall marker bars at each end and breaking the remaining bars into six or ten groups of five. In each group the five bars signify (from left to right) the values $7, 4, 2, 1, 0$. Exactly two of them will be tall. Their sum is the represented digit, except that zero is represented as $7 + 4$. The five- or nine-digit zip code is followed by a check digit, with the check equation

$$\sum d_i = 0 \pmod{10} \tag{22.4.6}$$

None of these schemes is close to optimal. An elegant scheme due to Verhoeff is described in [6]. The underlying idea is to use the ten-element *dihedral group* D_5, which corresponds to the symmetries of a pentagon, instead of the cyclic group of the integers modulo 10. The check equation is

$$a_1 * f(a_2) * f^2(a_3) * \cdots * f^{n-1}(a_n) = 0 \tag{22.4.7}$$

where $*$ is (noncommutative) multiplication in D_5, and f^i denotes the ith iteration of a certain fixed permutation. Verhoeff's method finds *all* single errors in a string, and *all* adjacent transpositions. It also finds about 95% of twin errors ($aa \to bb$), jump transpositions ($acb \to bca$), and jump twin errors ($aca \to bcb$). Here is an implementation:

decchk.h

```
Bool decchk(string str, char &ch) {
```

Decimal check digit computation or verification. Returns as `ch` a check digit for appending to `string[0..n-1]`, that is, for storing into `string[n]`. In this mode, ignore the returned boolean value. If `string[0..n-1]` already ends with a check digit (`string[n-1]`), returns the function value `true` if the check digit is valid, otherwise `false`. In this mode, ignore the returned value of `ch`. Note that `string` and `ch` contain ASCII characters corresponding to the digits 0-9, *not*

byte values in that range. Other ASCII characters are allowed in `string`, and are ignored in calculating the check digit.

```
    char c;
    Int j,k=0,m=0,n=str.length();
    static Int ip[10][8]={{0,1,5,8,9,4,2,7},{1,5,8,9,4,2,7,0},
        {2,7,0,1,5,8,9,4},{3,6,3,6,3,6,3,6},{4,2,7,0,1,5,8,9},
        {5,8,9,4,2,7,0,1},{6,3,6,3,6,3,6,3},{7,0,1,5,8,9,4,2},
        {8,9,4,2,7,0,1,5},{9,4,2,7,0,1,5,8}};
    static Int ij[10][10]={{0,1,2,3,4,5,6,7,8,9},{1,2,3,4,0,6,7,8,9,5},
        {2,3,4,0,1,7,8,9,5,6},{3,4,0,1,2,8,9,5,6,7},{4,0,1,2,3,9,5,6,7,8},
        {5,9,8,7,6,0,4,3,2,1},{6,5,9,8,7,1,0,4,3,2},{7,6,5,9,8,2,1,0,4,3},
        {8,7,6,5,9,3,2,1,0,4},{9,8,7,6,5,4,3,2,1,0}};
        Group multiplication and permutation tables.
    for (j=0;j<n;j++) {                  Look at successive characters.
        c=str[j];
        if (c >= 48 && c <= 57)          Ignore everything except digits.
            k=ij[k][ip[(c+2) % 10][7 & m++]];
    }
    for (j=0;j<10;j++)                   Find which appended digit will check properly.
        if (ij[k][ip[j][m & 7]] == 0) break;
    ch=char(j+48);                       Convert to ASCII.
    return k==0;
}
```

CITED REFERENCES AND FURTHER READING:

Saadawi, T.N., and Ammar, M.H. 1994, *Fundamentals of Telecommunication Networks* (New York: Wiley).[1]

LeVan, J. 1987, "A Fast CRC," *Byte*, vol. 12, pp. 339–341 (November).[2]

Koopman, P., and Chakravarty, T. 2004, "Cyclic Redundancy Code (CRC) Polynomial Selection for Embedded Networks," in *International Conference on Dependable Systems and Networks (DSN-2004)* (IEEE Computer Society).[3]

Sarwate, D.V. 1988, "Computation of Cyclic Redundancy Checks via Table Look-Up," *Communications of the ACM*, vol. 31, pp. 1008–1013.[4]

Griffiths, G., and Stones, G.C. 1987, "The Tea-Leaf Reader Algorithm: An Efficient Implementation of CRC-16 and CRC-32," *Communications of the ACM*, vol. 30, pp. 617–620.[5]

Wagner, N.R., and Putter, P.S. 1989, "Error Detecting Decimal Digits," *Communications of the ACM*, vol. 32, pp. 106–110.[6]

22.5 Huffman Coding and Compression of Data

A lossless data compression algorithm takes a string of symbols (typically ASCII characters or bytes) and translates it *reversibly* into another string, one that is *on the average* of shorter length. The words "on the average" are crucial; it is obvious that no reversible algorithm can make all strings shorter — there just aren't enough short strings to be in one-to-one correspondence with longer strings. Compression algorithms are possible only when, on the input side, some strings, or some input symbols, are more common than others. These can then be encoded in fewer bits than rarer input strings or symbols, giving a net average gain. We already quantified this idea, with the concept of *entropy*, in §14.7.

There exist many, quite different, compression techniques, corresponding to different ways of detecting and using departures from equiprobability in input strings.

In this section and the next we shall consider only *variable length codes* with *defined word* inputs. In these, the input is sliced into fixed units, for example ASCII characters, while the corresponding output comes in chunks of variable size. The simplest such method is Huffman coding [1], discussed in this section. Another example, *arithmetic compression*, is discussed in §22.6.

At the opposite extreme from defined-word, variable length codes are schemes that divide up the *input* into units of variable length (words or phrases of English text, for example) and then transmit these, often with a fixed length output code. The most widely used code of this general type is the Ziv-Lempel code [2]. References [3-5] give the flavor of some other compression techniques, with references to the large literature.

The idea behind Huffman coding is simply to use shorter bit patterns for more common characters. Suppose the input alphabet has $N_{\rm ch}$ characters, and that these occur in the input string with respective probabilities p_i, $i = 1, \ldots, N_{\rm ch}$, so that $\sum p_i = 1$. As we saw in §14.7, strings consisting of independently random sequences of these characters (a conservative, but not always realistic assumption) require, on the average, at least

$$H = -\sum p_i \log_2 p_i \tag{22.5.1}$$

bits per character, where H is the entropy of the probability distribution. Moreover, coding schemes exist that approach the bound arbitrarily closely. For the case of equiprobable characters, with all $p_i = 1/N_{\rm ch}$, one easily sees that $H = \log_2 N_{\rm ch}$, which is the case of no compression at all. Any other set of p_i's gives a smaller entropy, allowing some useful compression.

Notice that the bound of (22.5.1) would be achieved if we could encode character i with a code of length $L_i = -\log_2 p_i$ bits: Equation (22.5.1) would then be the average $\sum p_i L_i$. The trouble with such a scheme is that $-\log_2 p_i$ is not generally an integer. How can we encode the letter "Q" in 5.32 bits? Huffman coding makes a stab at this by, in effect, approximating all the probabilities p_i by integer powers of 1/2, so that all the L_i's are integral. If all the p_i's are in fact of this form, then a Huffman code does achieve the entropy bound H.

The construction of a Huffman code is best illustrated by example. Imagine a language, Vowellish, with the $N_{\rm ch} = 5$ character alphabet A, E, I, O, and U, occurring with the respective probabilities 0.12, 0.42, 0.09, 0.30, and 0.07. Then the construction of a Huffman code for Vowellish is accomplished in the table on the next page.

Here is how it works, proceeding in sequence through $N_{\rm ch}$ stages, represented by the columns of the table. The first stage starts with $N_{\rm ch}$ nodes, one for each letter of the alphabet, containing their respective relative frequencies. At each stage, the two smallest probabilities are found, summed to make a new node, and then dropped from the list of active nodes. (A "block" denotes the stage where a node is dropped.) All active nodes (including the new composite) are then carried over to the next stage (column). In the table, the names assigned to new nodes (e.g., AUI) are inconsequential. In the example shown, it happens that (after stage 1) the two smallest nodes are always an original node and a composite one; this need not be true in general: The two smallest probabilities might be both original nodes, or both composites, or one of each. At the last stage, all nodes will have been collected into one grand composite of total probability 1.

Node	Stage:	1	2	3	4	5
1	A:	0.12	0.12 ■			
2	E:	0.42	0.42	0.42	0.42 ■	
3	I:	0.09 ■				
4	O:	0.30	0.30	0.30 ■		
5	U:	0.07 ■				
6		UI:	0.16 ■			
7			AUI:	0.28 ■		
8				AUIO:	0.58 ■	
9					EAUIO:	1.00

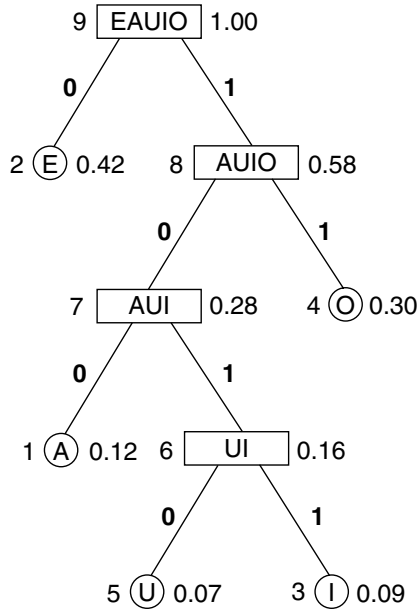


Figure 22.5.1. Huffman code for the fictitious language Vowellish, in tree form. A letter (A, E, I, O, or U) is encoded or decoded by traversing the tree from the top down; the code is the sequence of 0's and 1's on the branches. The value to the right of each node is its probability; to the left, its node number in the table.

Now, to see the code, you redraw the data in the table as a tree (Figure 22.5.1). As shown, each node of the tree corresponds to a node (row) in the table, indicated by the integer to its left and probability value to its right. Terminal nodes, so called, are shown as circles; these are single alphabetic characters. The branches of the tree are labeled 0 and 1. The code for a character is the sequence of zeros and ones that lead to it, from the top down. For example, E is simply 0, while U is 1010.

Any string of zeros and ones can now be decoded into an alphabetic sequence. Consider, for example, the string 1011111010. Starting at the top of the tree we

descend through 1011 to I, the first character. Since we have reached a terminal node, we reset to the top of the tree, next descending through 11 to O. Finally 1010 gives U. The string thus decodes to IOU.

These ideas are embodied in the following `Huffcode` object. The constructor lets you specify $N_{\rm ch}$, and an integer frequency-of-occurrence table of length $N_{\rm ch}$ telling how often each character occurs in some large corpus of text. These integers are, of course, proportional to the p_i's. The reason for using integers is so that any two computers will produce exactly the same code from the same input data. This might not be true if we used floating-point values. The constructor utilizes a heap structure (see §8.3) for efficiency; for a detailed description, see Sedgewick [6].

Once you have created an instance of `Huffcode`, you code a message by calling `codeone` for each message character in turn. This writes bits into a byte array `code` that you supply as an argument. There is no message-dependent saved state, so you could interleave different messages if there were some reason to do so.

Decoding a Huffman-encoded message is slightly more complicated. The coding tree must be traversed from the top down, using up a variable number of bits. This is done by the method `decodeone`.

There is no such thing as an "end of message" (EOM) marker in Huffman codes — not unless you provide one. Successive calls to `decodeone` will happily decode bits into characters until your hardware traps an illegal memory read! That is because every path on the tree (cf. Figure 22.5.1) terminates in a valid character. In practice, one increases $N_{\rm ch}$ by 1, and gives the extra character a frequency of occurrence of 1 (versus large values for the other characters). The new character becomes the EOM marker. Similarly, one can add other extra characters for other "out-of-band" signaling. If these occur rarely, the overhead on the message is negligible.

huffcode.h

```
struct Huffcode {
```

Object for Huffman encoding and decoding.

```
    Int nch,nodemax,mq;
    Int ilong,nlong;
    VecInt ncod,left,right;
    VecUint icod;
    Uint setbit[32];

    Huffcode(const Int nnch, VecInt_I &nfreq)
    : nch(nnch), mq(2*nch-1), icod(mq), ncod(mq), left(mq), right(mq) {
```

Constructor. Given the frequency of occurrence table `nfreq[0..nnch-1]` for `nnch` characters, constructs the Huffman code. Also sets `ilong` and `nlong` as the character number that produced the longest code symbol, and the length of that symbol.

```
        Int ibit,j,node,k,n,nused;
        VecInt index(mq), nprob(mq), up(mq);
        for (j=0;j<32;j++) setbit[j] = 1 << j;
        for (nused=0,j=0;j<nch;j++) {
            nprob[j]=nfreq[j];
            icod[j]=ncod[j]=0;
            if (nfreq[j] != 0) index[nused++]=j;
        }
        for (j=nused-1;j>=0;j--)
            heep(index,nprob,nused,j);
        k=nch;
        while (nused > 1) {
            node=index[0];
            index[0]=index[(nused--)-1];
            heep(index,nprob,nused,0);
            nprob[k]=nprob[index[0]]+nprob[node];
```

Sort `nprob` into a heap structure in `index`.

Combine heap nodes, remaking the heap at each stage.

```
            left[k]=node;                    Store left and right children of a node.
            right[k++]=index[0];
            up[index[0]] = -Int(k);          Indicate whether a node is a left or right child
            index[0]=k-1;                        of its parent.
            up[node]=k;
            heep(index,nprob,nused,0);
        }
        up[(nodemax=k)-1]=0;
        for (j=0;j<nch;j++) {                Make the Huffman code from the tree.
            if (nprob[j] != 0) {
                for (n=0,ibit=0,node=up[j];node;node=up[node-1],ibit++) {
                    if (node < 0) {
                        n |= setbit[ibit];
                        node = -node;
                    }
                }
                icod[j]=n;
                ncod[j]=ibit;
            }
        }
        nlong=0;
        for (j=0;j<nch;j++) {
            if (ncod[j] > nlong) {
                nlong=ncod[j];
                ilong=j;
            }
        }
        if (nlong > numeric_limits<Uint>::digits)
            throw("Code too long in Huffcode.  See text.");
    }
```

```
    void codeone(const Int ich, char *code, Int &nb) {
```

Huffman encode the single character `ich` (in the range `0..nch-1`), write the result to the byte array `code` starting at bit `nb` (whose smallest valid value is zero), and increment `nb` to the first unused bit. This routine is called repeatedly to encode consecutive characters in a message. The user is responsible for monitoring that the value of `nb` does not overrun the length of `code`.

```
        Int m,n,nc;
        if (ich >= nch) throw("bad ich (out of range) in Huffcode");
        if (ncod[ich]==0) throw("bad ich (zero prob) in Huffcode");
        for (n=ncod[ich]-1;n >= 0;n--,++nb) {        Loop over the bits in the stored
            nc=nb >> 3;                                  Huffman code for ich.
            m=nb & 7;
            if (m == 0) code[nc]=0;                  Set appropriate bits in code.
            if ((icod[ich] & setbit[n]) != 0) code[nc] |= setbit[m];
        }
    }
```

```
    Int decodeone(char *code, Int &nb) {
```

Starting at bit number `nb` in the byte array `code`, decode a single character (returned as `ich` in the range `0..nch-1`) and increment `nb` appropriately. Repeated calls, starting with `nb` = 0, will return successive characters in a compressed message. The user is responsible for detecting EOM from the message content.

```
        Int nc;
        Int node=nodemax-1;
        for (;;) {                               Set node to the top of the decoding tree, and
            nc=nb >> 3;                              loop until a valid character is obtained.
            node=((code[nc] & setbit[7 & nb++]) != 0 ?
                right[node] : left[node]);
                Branch left or right in tree, depending on its value.
            if (node < nch) return node;         If we reach a terminal node, we have a com-
        }                                            plete character and can return.
    }
```

```
    void heep(VecInt_IO &index, VecInt_IO &nprob, const Int n, const Int m) {
    Used by the constructor to maintain a heap structure in the array index[0..m-1].
        Int i=m,j,k;
        k=index[i];
        while (i < (n >> 1)) {
            if ((j = 2*i+1) < n-1
                && nprob[index[j]] > nprob[index[j+1]]) j++;
            if (nprob[k] <= nprob[index[j]]) break;
            index[i]=index[j];
            i=j;
        }
        index[i]=k;
    }
};
```

`Huffcode` requires that the longest code for a single character fits into your machine's integer wordlength (typically 32 bits), and will tell you if this is violated. If this happens, you'll need to increase the frequency-of-occurrence value for the rarest characters. This will affect your compression hardly at all.

It is a feature, not a bug, that `Huffcode` allows you to specify some characters as having zero frequency of occurrence, and then completely omits these from the code. This can be very useful when, for example, you want to compress a file consisting only of ASCII characters 0–9, +, −, and ".", as might occur in a file of numerical values. But don't then try to encode one of the omitted characters!

22.5.1 Run-Length Encoding

For the compression of highly correlated bit streams (for example the black or white values along a facsimile scan line), Huffman compression is often combined with *run-length encoding*: Instead of sending each bit, the input stream is converted to a series of integers indicating how many consecutive bits have the same value. These integers are then Huffman-compressed. The Group 3 CCITT facsimile standard functions in this manner, with a fixed, immutable, Huffman code, optimized for a set of eight standard documents [7].

CITED REFERENCES AND FURTHER READING:

Hamming, R.W. 1980, *Coding and Information Theory* (Englewood Cliffs, NJ: Prentice-Hall).

Huffman, D.A. 1952, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the Institute of Radio Engineers*, vol. 40, pp. 1098–1101.[1]

Ziv, J., and Lempel, A. 1978, "Compression of Individual Sequences via Variable-Rate Coding," *IEEE Transactions on Information Theory*, vol. IT-24, pp. 530–536.[2]

Sayood, K. 2005, *Introduction to Data Compression*, 3rd ed. (San Francisco: Morgan Kaufmann).[3]

Salomon, D. 2004, *Data Compression: The Complete Reference*, 3rd ed. (New York: Springer).[4]

Wayner, P. 1999, *Compression Algorithms for Real Programmers* (San Francisco: Morgan Kaufmann).[5]

Sedgewick, R. 1998, *Algorithms in C*, 3rd ed. (Reading, MA: Addison-Wesley), Chapter 22.[6]

Hunter, R., and Robinson, A.H. 1980, "International Digital Facsimile Coding Standard," *Proceedings of the IEEE*, vol. 68, pp. 854–867.[7]

22.6 Arithmetic Coding

We saw in the previous section, as well as §14.7, that a perfect coding scheme would use $L_i = -\log_2 p_i$ bits to encode character i (in the range $1 \le i \le N_{\rm ch}$), if p_i is its probability of occurrence and characters occur independently randomly. Huffman coding gives a way of rounding the L_i's to close integer values and constructing a code with those lengths. *Arithmetic coding* [1], which we now discuss, actually does manage to encode characters using noninteger numbers of bits! It also provides a convenient way to output the result not as a stream of bits, but as a stream of symbols in any desired radix. This latter property is particularly useful if you want, e.g., to convert data from bytes (radix 256) to printable ASCII characters (radix 94), or to case-independent alphanumeric sequences containing only A-Z and 0-9 (radix 36).

In arithmetic coding, an input message of any length is represented as a real number R in the range $0 \le R < 1$. The longer the message, the more precision required of R. This is best illustrated by an example, so let us return to the fictitious language, Vowellish, of the previous section. Recall that Vowellish has a five-character alphabet (A, E, I, O, U), with occurrence probabilities 0.12, 0.42, 0.09, 0.30, and 0.07, respectively. Figure 22.6.1 shows how a message beginning "IOU" is encoded: The interval $[0, 1)$ is divided into segments corresponding to the five alphabetical characters; the length of a segment is the corresponding p_i. We see that the first message character, "I", narrows the range of R to $0.37 \le R < 0.46$. This interval is now subdivided into five subintervals, again with lengths proportional to the p_i's. The second message character, "O", narrows the range of R to $0.3763 \le R < 0.4033$. The "U" character further narrows the range to $0.37630 \le R < 0.37819$. *Any* value of R in this range can be sent as encoding "IOU". In particular, the binary fraction .011000001 is in this range, so "IOU" can be sent in 9 bits. (Huffman coding took 10 bits for this example; see §22.5.)

Of course there is the problem of knowing when to stop decoding. The fraction .011000001 represents not simply "IOU," but "IOU...," where the ellipses represent an infinite string of successor characters. We had a similar problem in Huffman coding, but there we would at least stop when we ran off the edge of the input buffer. Here, the real number .011000001 actually does represent an infinite message! Arithmetic coding therefore *always* must assume the existence of a special $N_{\rm ch} + 1$st character, EOM (end of message), which occurs only once at the end of the input. Since EOM has a low probability of occurrence, it gets allocated only a very tiny piece of the number line.

In the above example, we gave R as a binary fraction. We could just as well have output it in any other radix, e.g., base 94 or base 36, whatever is convenient for the anticipated storage or communication channel.

You might wonder how one deals with the seemingly incredible precision required of R for a long message. The answer is that R is never actually represented all at once. At any give stage we have upper and lower bounds for R represented as a finite number of digits in the output radix. As digits of the upper and lower bounds become identical, we can left-shift them away and bring in new digits at the low-significance end. The object below has a parameter `NWK` for the number of working digits to keep around. This must be large enough to make the chance of an accidental degeneracy vanishingly small. (The object signals if a degeneracy ever occurs.)

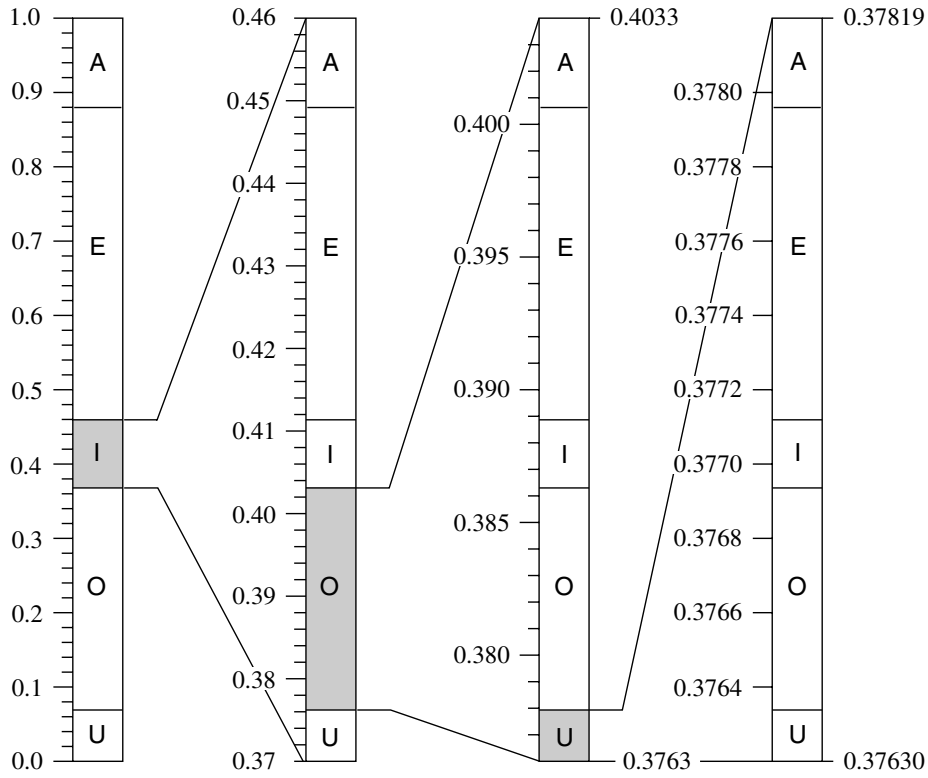


Figure 22.6.1. Arithmetic coding of the message "IOU..." in the fictitious language Vowellish. Successive characters give successively finer subdivisions of the initial interval between 0 and 1. The final value can be output as the digits of a fraction in any desired radix. Note how the subinterval allocated to a character is proportional to its probability of occurrence.

Since the process of discarding old digits and bringing in new ones is performed identically on encoding and decoding, everything stays synchronized.

In the `Arithcode` object below, the constructor has arguments to specify the number of characters and an integer frequency-of-occurrence table (as in `Huffcode`), plus an argument that allows you to specify an output radix for the code. Because there is some saved state between coding successive characters (the upper and lower bounds for R, for example), you must call `messageinit` prior to encoding or decoding the first character of a new message, and not interleave the encoding of different messages in a single instance of `Arithcode`. If you want to interleave messages, create more than one instance.

Successive calls to `codeone` for each input character encode the message. A final call with character `nch` (that is, one larger than your specified character set) adds the EOM marker and is mandatory. After this final call, `lcd` will be set to the number of bytes in the coded message (i.e., will point to the first unused location in `code`). The `decodeone` routine similarly returns successive characters of the decoded message in turn, with `nch` returned to indicate EOM.

Unlike the `Huffcode` object, `Arithcode` has no provision for omitting specified message characters from the code. Therefore, it also refuses to believe zero val-

ues in the table nfreq; a 0 is treated as if it were a 1. If you want to live dangerously, with a very slightly more efficient coding, you can change this in the constructor.

arithcode.h

```
struct Arithcode {
```
Object for arithmetic coding.
```
    Int nch,nrad,ncum;
    Uint jdif,nc,minint;
    VecUint ilob,iupb;
    VecInt ncumfq;
    static const Int NWK=20;
```
Number of working digits.
```
    Arithcode(VecInt_I &nfreq, const Int nnch, const Int nnrad)
    : nch(nnch), nrad(nnrad), ilob(NWK), iupb(NWK), ncumfq(nch+2) {
```
Constructor. Given the frequency of occurrence table nfreq[0..nnch-1] for nnch characters, constructs the Huffman code whose output has radix nnrad (which must be ≤ 256).
```
        Int j;
        if (nrad > 256) throw("output radix must be <= 256 in Arithcode");
        minint=numeric_limits<Uint>::max()/nrad;
        ncumfq[0]=0;
        for (j=1;j<=nch;j++) ncumfq[j]=ncumfq[j-1]+MAX(nfreq[j-1],1);
        ncum=ncumfq[nch+1]=ncumfq[nch]+1;
    }

    void messageinit() {
```
Clear saved state for a new message (either encode or decode). This is mandatory before encoding or decoding the first character.
```
        Int j;
        jdif=nrad-1;
        for (j=NWK-1;j>=0;j--) {
            iupb[j]=nrad-1;
            ilob[j]=0;
            nc=j;
            if (jdif > minint) return;
            jdif=(jdif+1)*nrad-1;
        }
        throw("NWK too small in arcode.");
    }
```
Initialize enough digits of the upper and lower bounds.

Initialization complete.
```
    void codeone(const Int ich, char *code, Int &lcd) {
```
Encode the single character ich in the range $0 \ldots$ nch-1 into the byte array code, starting at location code[lcd] and (if necessary) incrementing lcd so that, on return, it points to the first unused byte in code. A final call with ich=nch encodes "end of message." Byte values written into code will be in the range $0 \ldots \text{nrad} - 1$.
```
        if (ich > nch) throw("bad ich in Arithcode");
        advance(ich,code,lcd,1);
    }
```
Check for valid input character.
```
    Int decodeone(char *code, Int &lcd) {
```
Decode and return a single message character, using code starting at location code[lcd], and (if necessary) increment lcd appropriately. Successive calls return successive message characters. The returned value nch indicates end of message (subsequent calls will return nonsense).
```
        Int ich;
        Uint j,ihi,ja,m;
        ja=(Uchar) code[lcd]-ilob[nc];
        for (j=nc+1;j<NWK;j++) {
            ja *= nrad;
            ja += Uchar(code[lcd+j-nc])-ilob[j];
        }
        ihi=nch+1;
        ich=0;
        while (ihi-ich > 1) {
```
If decoding, locate the character ich by bisection.

```
            m=(ich+ihi)>>1;
            if (ja >= multdiv(jdif,ncumfq[m],ncum)) ich=m;
            else ihi=m;
        }
        if (ich != nch) advance(ich,code,lcd,-1);
        return ich;
    }

    void advance(const Int ich, char *code, Int &lcd, const Int isign) {
```

Used internally. Operations common to encoding and decoding. Convert character `ich` to a new subrange `[ilob,iupb)`.

```
        Uint j,k,jh,jl;
        jh=multdiv(jdif,ncumfq[ich+1],ncum);
        jl=multdiv(jdif,ncumfq[ich],ncum);
        jdif=jh-jl;
        arrsum(ilob,iupb,jh,NWK,nrad,nc);
        arrsum(ilob,ilob,jl,NWK,nrad,nc);
        for (j=nc;j<NWK;j++) {
            if (ich != nch && iupb[j] != ilob[j]) break;
            if (isign > 0) code[lcd] = ilob[j];
            lcd++;
        }
        if (j+1 > NWK) return;
        nc=j;
        for(j=0;jdif<minint;j++)
            jdif *= nrad;
        if (j > nc) throw("NWK too small in arcode.");
        if (j != 0) {
            for (k=nc;k<NWK;k++) {
                iupb[k-j]=iupb[k];
                ilob[k-j]=ilob[k];
            }
        }
        nc -= j;
        for (k=NWK-j;k<NWK;k++) iupb[k]=ilob[k]=0;
        return;
    }

    inline Uint multdiv(const Uint j, const Uint k, const Uint m) {
```

Calculate `(k*j)/m` without overflow by use of double-length integers.

```
        return Uint((Ullong(j)*Ullong(k)/Ullong(m)));
    }

    void arrsum(VecUint_I &iin, VecUint_O &iout, Uint ja,
    const Int nwk, const Uint nrad, const Uint nc) {
```

Add the integer `ja` to the radix `nrad` multiple-precision integer `iin[nc..nwk-1]`. Return the result in `iout[nc..nwk-1]`.

```
        Uint karry=0,j,jtmp;
        for (j=nwk-1;j>nc;j--) {
            jtmp=ja;
            ja /= nrad;
            iout[j]=iin[j]+(jtmp-ja*nrad)+karry;
            if (iout[j] >= nrad) {
                iout[j] -= nrad;
                karry=1;
            } else karry=0;
        }
        iout[nc]=iin[nc]+ja+karry;
    }
};
```

How many leading digits to output (if encoding) or skip over?

Ran out of message. Did someone forget to encode a terminating `ncd`?

How many digits to shift?

Shift them.

Normal return.

A few further notes: When an interval of size `jdif` is to be partitioned in the proportions of some `n` to some `ntot`, say, then we must compute `(n*jdif)/ntot`.

With ordinary integer arithmetic, the numerator is likely to overflow; and, unfortunately, an expression like `jdif/(ntot/n)` is not equivalent. We therefore need to use double-length integers, type `Ullong`, usually 64 bits, just for this operation.

The internally set variable `minint`, which is the minimum allowed number of discrete steps between the upper and lower bounds, determines when new low-significance digits are added. `minint` must be large enough to provide resolution of all the input characters. That is, we must have $p_i \times \texttt{minint} > 1$ for all i. A value of $100N_{\rm ch}$, or $1.1/\min p_i$, whichever is larger, is generally adequate. However, for safety, the routine takes `minint` to be as large as possible, with the product `minint*nradd` just smaller than overflow. This results in some time inefficiency, and in a few unnecessary characters being output at the end of a message. You can decrease `minint` if you want to live closer to the edge.

If radix-changing, rather than compression, is your primary aim (for example to convert an arbitrary file into printable characters), then you are of course free to set all the components of `nfreq` equal, say, to 1.

While the output radix is limited to 256 (so that values fit into a byte), the input alphabet size $N_{\rm ch} =$ `nch` can be less than, equal to, or greater than 256.

CITED REFERENCES AND FURTHER READING:

Sayood, K. 2005, *Introduction to Data Compression*, 3rd ed. (San Francisco: Morgan Kaufmann).

Salomon, D. 2004, *Data Compression: The Complete Reference*, 3rd ed. (New York: Springer).

Wayner, P. 1999, *Compression Algorithms for Real Programmers* (San Francisco: Morgan Kaufmann).

Witten, I.H., Neal, R.M., and Cleary, J.G. 1987, "Arithmetic Coding for Data Compression," *Communications of the ACM*, vol. 30, pp. 520–540.[1]

22.7 Arithmetic at Arbitrary Precision

Let's compute the number π to a couple of thousand decimal places. In doing so, we'll learn some things about multiple precision arithmetic on computers and meet quite an unusual application of the fast Fourier transform (FFT). We'll also develop a set of routines that you can use for other calculations at any desired level of arithmetic precision.

To start with, we need an analytic algorithm for π. Useful algorithms are quadratically convergent, i.e., they double the number of significant digits at each iteration. Quadratically convergent algorithms for π are based on the *AGM (arithmetic geometric mean)* method, which also finds application to the calculation of elliptic integrals (cf. §6.12) and in advanced implementations of the ADI method for elliptic partial differential equations (§20.5). Borwein and Borwein [1] treat this subject, which is beyond our scope here. One of their algorithms for π starts with the initializations

$$\begin{aligned} X_0 &= \sqrt{2} \\ \pi_0 &= 2 + \sqrt{2} \\ Y_0 &= \sqrt[4]{2} \end{aligned} \tag{22.7.1}$$

and then, for $i = 0, 1, \ldots,$ repeats the iteration

$$\begin{aligned} X_{i+1} &= \frac{1}{2}\left(\sqrt{X_i} + \frac{1}{\sqrt{X_i}}\right) \\ \pi_{i+1} &= \pi_i \left(\frac{X_{i+1}+1}{Y_i+1}\right) \\ Y_{i+1} &= \frac{Y_i\sqrt{X_{i+1}} + \dfrac{1}{\sqrt{X_{i+1}}}}{Y_i+1} \end{aligned} \tag{22.7.2}$$

The value π emerges as the limit π_∞.

Now to the question of how to do arithmetic to arbitrary precision: In a high-level language like C++, a natural choice is to work in radix (base) 256, so that character arrays can be directly interpreted as strings of digits. At the very end of our calculation, we will want to convert our answer to radix 10, but that is essentially a frill for the benefit of human ears, accustomed to the familiar chant, "three point one four one five nine. . . ." For any less frivolous calculation, we would likely never leave base 256 (or the thence trivially reachable hexadecimal, octal, or binary bases).

We will adopt the convention of storing digit strings in the "human" ordering, that is, with the first stored digit in an array being most significant, the last stored digit being least significant. The opposite convention would, of course, also be possible. "Carries," where we need to partition a number larger than 255 into a low-order byte and a high-order carry, present a minor programming annoyance, solved, in the routines below, by the use of the functions `lobyte` and `hibyte`. It will be our usual convention to assume that the digit strings represent floating-point numbers with the radix point falling after the the first digit. When an operation results in a number that requires more digits in front of the decimal point, it is the responsibility of the user to shift the digits to the right and keep track of any excess factors of 256 that this implies.

It is easy at this point, following Knuth [2], to write a routines for the "fast" arithmetic operations: short addition (adding a single byte to a string), addition, subtraction, short multiplication (multiplying a string by a single byte), short division, ones-complement negation, and a couple of utility operations, copying and left-shifting strings. These are implemented in the following `MParith` object. The additional routines that are declared, but not defined, are discussed below.

mparith.h

```
struct MParith {
```

Multiple precision arithmetic operations done on character strings, interpreted as radix 256 numbers with the radix point after the first digit. Implementations for the simpler operations are listed here.

```
    void mpadd(VecUchar_O &w, VecUchar_I &u, VecUchar_I &v) {
```

Adds the unsigned radix 256 numbers `u` and `v`, yielding the unsigned result `w`. To achieve the full available accuracy, the array `w` must be longer, by one element, than the shorter of the two arrays `u` and `v`.

```
        Int j,n=u.size(),m=v.size(),p=w.size();
        Int n_min=MIN(n,m),p_min=MIN(n_min,p-1);
        Uint ireg=0;
        for (j=p_min-1;j>=0;j--) {
            ireg=u[j]+v[j]+hibyte(ireg);
            w[j+1]=lobyte(ireg);
        }
        w[0]=hibyte(ireg);
        if (p > p_min+1)
```

```
        for (j=p_min+1;j<p;j++) w[j]=0;
}

void mpsub(Int &is, VecUchar_O &w, VecUchar_I &u, VecUchar_I &v) {
```

Subtracts the unsigned radix 256 number `v` from `u` yielding the unsigned result `w`. If the result is negative (wraps around), `is` is returned as -1; otherwise it is returned as 0. To achieve the full available accuracy, the array `w` must be as long as the shorter of the two arrays `u` and `v`.

```
    Int j,n=u.size(),m=v.size(),p=w.size();
    Int n_min=MIN(n,m),p_min=MIN(n_min,p-1);
    Uint ireg=256;
    for (j=p_min-1;j>=0;j--) {
        ireg=255+u[j]-v[j]+hibyte(ireg);
        w[j]=lobyte(ireg);
    }
    is=hibyte(ireg)-1;
    if (p > p_min)
        for (j=p_min;j<p;j++) w[j]=0;
}

void mpsad(VecUchar_O &w, VecUchar_I &u, const Int iv) {
```

Short addition: The integer `iv` (in the range $0 \le$ `iv` ≤ 255) is added to the least significant radix position of unsigned radix 256 number `u`, yielding result `w`. To ensure that the result does not require two digits before the radix point, one may first right-shift the operand `u` so that the first digit is 0, and keep track of multiples of 256 separately.

```
    Int j,n=u.size(),p=w.size();
    Uint ireg=256*iv;
    for (j=n-1;j>=0;j--) {
        ireg=u[j]+hibyte(ireg);
        if (j+1 < p) w[j+1]=lobyte(ireg);
    }
    w[0]=hibyte(ireg);
    for (j=n+1;j<p;j++) w[j]=0;
}

void mpsmu(VecUchar_O &w, VecUchar_I &u, const Int iv) {
```

Short multiplication: The unsigned radix 256 number `u` is multiplied by the integer `iv` (in the range $0 \le$ `iv` ≤ 255), yielding result `w`. To ensure that the result does not require two digits before the radix point, one may first right-shift the operand `u` so that the first digit is 0, and keep track of multiples of 256 separately.

```
    Int j,n=u.size(),p=w.size();
    Uint ireg=0;
    for (j=n-1;j>=0;j--) {
        ireg=u[j]*iv+hibyte(ireg);
        if (j < p-1) w[j+1]=lobyte(ireg);
    }
    w[0]=hibyte(ireg);
    for (j=n+1;j<p;j++) w[j]=0;
}

void mpsdv(VecUchar_O &w, VecUchar_I &u, const Int iv, Int &ir) {
```

Short division: The unsigned radix 256 number `u` is divided by the integer `iv` (in the range $0 \le$ `iv` ≤ 255), yielding a quotient `w` and a remainder `ir` (with $0 \le$ `ir` ≤ 255). To achieve the full available accuracy, the array `w` must be as long as the array `u`.

```
    Int i,j,n=u.size(),p=w.size(),p_min=MIN(n,p);
    ir=0;
    for (j=0;j<p_min;j++) {
        i=256*ir+u[j];
        w[j]=Uchar(i/iv);
        ir=i % iv;
    }
    if (p > p_min)
        for (j=p_min;j<p;j++) w[j]=0;
}
```

```
    void mpneg(VecUchar_IO &u) {
    Ones-complement negate the unsigned radix 256 number u.
        Int j,n=u.size();
        Uint ireg=256;
        for (j=n-1;j>=0;j--) {
            ireg=255-u[j]+hibyte(ireg);
            u[j]=lobyte(ireg);
        }
    }

    void mpmov(VecUchar_O &u, VecUchar_I &v) {
    Move the unsigned radix 256 number v into u. To achieve full accuracy, the array v must
    be as long as the array u.
        Int j,n=u.size(),m=v.size(),n_min=MIN(n,m);
        for (j=0;j<n_min;j++) u[j]=v[j];
        if (n > n_min)
            for(j=n_min;j<n-1;j++) u[j]=0;
    }

    void mplsh(VecUchar_IO &u) {
    Left-shift digits of unsigned radix 256 number u. The final element of the array is set to 0.

        Int j,n=u.size();
        for (j=0;j<n-1;j++) u[j]=u[j+1];
        u[n-1]=0;
    }

    Uchar lobyte(Uint x) {return (x & 0xff);}
    Uchar hibyte(Uint x) {return ((x >> 8) & 0xff);}

    The following, more complicated, methods have discussion and implementation below.
    void mpmul(VecUchar_O &w, VecUchar_I &u, VecUchar_I &v);
    void mpinv(VecUchar_O &u, VecUchar_I &v);
    void mpdiv(VecUchar_O &q, VecUchar_O &r, VecUchar_I &u, VecUchar_I &v);
    void mpsqrt(VecUchar_O &w, VecUchar_O &u, VecUchar_I &v);
    void mp2dfr(VecUchar_IO &a, string &s);
    string mppi(const Int np);
};
```

Full multiplication of two strings of digits, if done by the traditional hand method, is not a fast operation: In multiplying two strings of length N, the multiplicand would be short-multiplied in turn by each byte of the multiplier, requiring $O(N^2)$ operations in all. We will see, however, that *all* the arithmetic operations on numbers of length N can in fact be done in $O(N \times \log N \times \log\log N)$ operations.

The trick is to recognize that multiplication is essentially a *convolution* (§13.1) of the digits of the multiplicand and multiplier, followed by some kind of carry operation. Consider, for example, two ways of writing the calculation 456×789:

$$\begin{array}{r} 456 \\ \times\ 789 \\ \hline 4104 \\ 3648 \\ 3192 \\ \hline 359784 \end{array} \qquad\qquad \begin{array}{rrrrrr} & & 4 & 5 & 6 & \\ \times & & 7 & 8 & 9 & \\ \hline & & 36 & 45 & 54 & \\ & 32 & 40 & 48 & & \\ 28 & 35 & 42 & & & \\ \hline 28 & 67 & 118 & 93 & 54 & \\ \hline 3 & 5 & 9 & 7 & 8 & 4 \end{array}$$

The tableau on the left shows the conventional method of multiplication, in which three separate short multiplications of the full multiplicand (by 9, 8, and 7) are added

to obtain the final result. The tableau on the right shows a different method (sometimes taught for mental arithmetic), where the single-digit cross products are all computed (e.g. $8 \times 6 = 48$), then added in columns to obtain an incompletely carried result (here, the list 28, 67, 118, 93, 54). The final step is a single pass from right to left, recording the single least-significant digit and carrying the higher digit or digits into the total to the left (e.g. $93 + 5 = 98$, record the 8, carry 9).

You can see immediately that the column sums in the right-hand method are components of the convolution of the digit strings, for example $118 = 4 \times 9 + 5 \times 8 + 6 \times 7$. In §13.1, we learned how to compute the convolution of two vectors by the fast Fourier transform (FFT): Each vector is FFT'd, the two complex transforms are multiplied, and the result is inverse-FFT'd. Since the transforms are done with floating arithmetic, we need sufficient precision so that the exact integer value of each component of the result is discernible in the presence of roundoff error. We should therefore allow a (conservative) few times $\log_2(\log_2 N)$ bits for roundoff in the FFT. A number of length N bytes in radix 256 can generate convolution components as large as the order of $(256)^2 N$, thus requiring $16 + \log_2 N$ bits of precision for exact storage. If `it` is the number of bits in the floating mantissa (cf. §22.2), we obtain the condition

$$16 + \log_2 N + \text{few} \times \log_2 \log_2 N < \texttt{it} \tag{22.7.3}$$

We see that single precision, say with $\texttt{it} = 24$, is inadequate for any interesting value of N, while double precision, say with $\texttt{it} = 53$, allows N to be greater than 10^6, corresponding to some millions of decimal digits. The use of `Doub` in the routines `realft` (§12.3) and `four1` (§12.2) is therefore a necessity, not merely a convenience, for this application.

mparith.h

```
void MParith::mpmul(VecUchar_O &w, VecUchar_I &u, VecUchar_I &v) {
```
Uses fast Fourier transform to multiply the unsigned radix 256 integers `u[0..n-1]` and `v[0..m-1]`, yielding a product `w[0..n+m-1]`.

```
    const Doub RX=256.0;
    Int j,nn=1,n=u.size(),m=v.size(),p=w.size(),n_max=MAX(m,n);
    Doub cy,t;
    while (nn < n_max) nn <<= 1;
    nn <<= 1;
    VecDoub a(nn,0.0),b(nn,0.0);
    for (j=0;j<n;j++) a[j]=u[j];
    for (j=0;j<m;j++) b[j]=v[j];
    realft(a,1);
    realft(b,1);
    b[0] *= a[0];
    b[1] *= a[1];
    for (j=2;j<nn;j+=2) {
        b[j]=(t=b[j])*a[j]-b[j+1]*a[j+1];
        b[j+1]=t*a[j+1]+b[j+1]*a[j];
    }
    realft(b,-1);
    cy=0.0;
    for (j=nn-1;j>=0;j--) {
        t=b[j]/(nn >> 1)+cy+0.5;
        cy=Uint(t/RX);
        b[j]=t-cy*RX;
    }
    if (cy >= RX) throw("cannot happen in mpmul");
    for (j=0;j<p;j++) w[j]=0;
    w[0]=Uchar(cy);
    for (j=1;j<MIN(n+m,p);j++) w[j]=Uchar(b[j-1]);
}
```

Find the smallest usable power of 2 for the transform.

Move U and V to double precision floating arrays.

Perform the convolution: First, the two Fourier transforms.

Then multiply the complex results (real and imaginary parts).

Then do the inverse Fourier transform.

Make a final pass to do all the carries.

The 0.5 allows for roundoff error.

Copy answer to output.

With multiplication thus a "fast" operation, division is best performed by multiplying the dividend by the reciprocal of the divisor. The reciprocal of a value V is calculated by iteration of Newton's rule,

$$U_{i+1} = U_i(2 - VU_i) \tag{22.7.4}$$

which results in the quadratic convergence of U_∞ to $1/V$, as you can easily prove. (Many historical supercomputers, and some more recent RISC processors, actually use this iteration to perform divisions.) We can now see where the operations count $N \log N \log\log N$, mentioned above, originates: $N \log N$ is in the Fourier transform, with the iteration to converge Newton's rule giving an additional factor of $\log\log N$.

mparith.h

```
void MParith::mpinv(VecUchar_O &u, VecUchar_I &v) {
```

Character string `v[0..m-1]` is interpreted as a radix 256 number with the radix point after (nonzero) `v[0]`; `u[0..n-1]` is set to the most significant digits of its reciprocal, with the radix point after `u[0]`.

```
    const Int MF=4;
    const Doub BI=1.0/256.0;
    Int i,j,n=u.size(),m=v.size(),mm=MIN(MF,m);
    Doub fu,fv=Doub(v[mm-1]);
    VecUchar s(n+m),r(2*n+m);
    for (j=mm-2;j>=0;j--) {
        fv *= BI;
        fv += v[j];
    }
    fu=1.0/fv;
    for (j=0;j<n;j++) {
        i=Int(fu);
        u[j]=Uchar(i);
        fu=256.0*(fu-i);
    }
    for (;;) {
        mpmul(s,u,v);
        mplsh(s);
        mpneg(s);
        s[0] += Uchar(2);
        mpmul(r,s,u);
        mplsh(r);
        mpmov(u,r);
        for (j=1;j<n-1;j++)
            if (s[j] != 0) break;
        if (j==n-1) return;
    }
}
```

Use ordinary floating arithmetic to get an initial approximation.

Iterate Newton's rule to convergence.

Construct $2 - UV$ in S.

Multiply SU into U.

If fractional part of S is not zero, it has not converged to 1.

Division now follows as a simple corollary, with only the necessity of calculating the reciprocal to sufficient accuracy to get an exact quotient and remainder.

mparith.h

```
void MParith::mpdiv(VecUchar_O &q, VecUchar_O &r, VecUchar_I &u, VecUchar_I &v) {
```

Divides unsigned radix 256 integers `u[0..n-1]` by `v[0..m-1]` (with `m` $\leq$ `n` required), yielding a quotient `q[0..n-m]` and a remainder `r[0..m-1]`.

```
    const Int MACC=1;
    Int i,is,mm,n=u.size(),m=v.size(),p=r.size(),n_min=MIN(m,p);
    if (m > n) throw("Divisor longer than dividend in mpdiv");
    mm=m+MACC;
    VecUchar s(mm),rr(mm),ss(mm+1),qq(n-m+1),t(n);
    mpinv(s,v);
    mpmul(rr,s,u);
    mpsad(ss,rr,1);
```

Set $S = 1/V$.

Set $Q = SU$.

```
    mplsh(ss);
    mplsh(ss);
    mpmov(qq,ss);
    mpmov(q,qq);
    mpmul(t,qq,v);                    Multiply and subtract to get the remainder.
    mplsh(t);
    mpsub(is,t,u,t);
    if (is != 0) throw("MACC too small in mpdiv");
    for (i=0;i<n_min;i++) r[i]=t[i+n-m];
    if (p>m) for (i=m;i<p;i++) r[i]=0;
}
```

Square roots are calculated by a Newton's rule much like division. If

$$U_{i+1} = \tfrac{1}{2}U_i(3 - VU_i^2) \tag{22.7.5}$$

then U_∞ converges quadratically to $1/\sqrt{V}$. A final multiplication by V gives $\sqrt{V}$.

```
void MParith::mpsqrt(VecUchar_O &w, VecUchar_O &u, VecUchar_I &v) {                mparith.h
```

Character string `v[0..m-1]` is interpreted as a radix 256 number with the radix point after `v[0]`; `w[0..n-1]` is set to its square root (radix point after `w[0]`), and `u[0..n-1]` is set to the reciprocal thereof (radix point before `u[0]`). `w` and `u` need not be distinct, in which case they are set to the square root.

```
    const Int MF=3;
    const Doub BI=1.0/256.0;
    Int i,ir,j,n=u.size(),m=v.size(),mm=MIN(m,MF);
    VecUchar r(2*n),x(n+m),s(2*n+m),t(3*n+m);
    Doub fu,fv=Doub(v[mm-1]);
    for (j=mm-2;j>=0;j--) {           Use ordinary floating arithmetic to get an initial ap-
        fv *= BI;                         proximation.
        fv += v[j];
    }
    fu=1.0/sqrt(fv);
    for (j=0;j<n;j++) {
        i=Int(fu);
        u[j]=Uchar(i);
        fu=256.0*(fu-i);
    }
    for (;;) {                        Iterate Newton's rule to convergence.
        mpmul(r,u,u);                 Construct S = (3 − VU^2)/2.
        mplsh(r);
        mpmul(s,r,v);
        mplsh(s);
        mpneg(s);
        s[0] += Uchar(3);
        mpsdv(s,s,2,ir);
        for (j=1;j<n-1;j++) {         If fractional part of S is not zero, it has not con-
            if (s[j] != 0) {              verged to 1.
                mpmul(t,s,u);         Replace U by SU.
                mplsh(t);
                mpmov(u,t);
                break;
            }
        }
        if (j<n-1) continue;
        mpmul(x,u,v);                 Get square root from reciprocal and return.
        mplsh(x);
        mpmov(w,x);
        return;
    }
}
```

We already mentioned that radix conversion to decimal is a merely cosmetic operation that should normally be omitted. The simplest way to convert a fraction to decimal is to multiply it repeatedly by 10, picking off (and subtracting) the resulting integer part. This has an operations count of $O(N^2)$, however, since each liberated decimal digit takes an $O(N)$ operation. It *is* possible to do the radix conversion as a fast operation by a "divide-and-conquer" strategy, in which the fraction is (fast) multiplied by a large power of 10, enough to move about half the desired digits to the left of the radix point. The integer and fractional pieces are now processed independently, each further subdivided. If our goal were a few billion digits of π, instead of a few thousand, we would need to implement this scheme. For present purposes, the following lazy routine is adequate:

mparith.h

```
void MParith::mp2dfr(VecUchar_IO &a, string &s)
```

Converts a radix 256 fraction `a[0..n-1]` (radix point before `a[0]`) to a decimal fraction represented as an ASCII string `s[0..m-1]`, where `m` is a returned value. The input array `a[0..n-1]` is destroyed. NOTE: For simplicity, this routine implements a slow ($\propto N^2$) algorithm. Fast ($\propto N \ln N$), more complicated, radix conversion algorithms do exist.

```
{
    const Uint IAZ=48;
    char buffer[4];
    Int j,m;

    Int n=a.size();
    m=Int(2.408*n);
    sprintf(buffer,"%d",a[0]);
    s=buffer;
    s += '.';
    mplsh(a);
    for (j=0;j<m;j++) {
        mpsmu(a,a,10);
        s += a[0]+IAZ;
        mplsh(a);
    }
}
```

Finally, then, we arrive at a routine implementing equations (22.7.1) and (22.7.2):

mparith.h

```
string MParith::mppi(const Int np) {
```

Demonstrate multiple precision routines by calculating and printing the first `np` bytes of π.

```
    const Uint IAOFF=48,MACC=2;
    Int ir,j,n=np+MACC;
    Uchar mm;
    string s;
    VecUchar x(n),y(n),sx(n),sxi(n),z(n),t(n),pi(n),ss(2*n),tt(2*n);
    t[0]=2;                                  Set T = 2.
    for (j=1;j<n;j++) t[j]=0;
    mpsqrt(x,x,t);                           Set X_0 = √2.
    mpadd(pi,t,x);                           Set π_0 = 2 + √2.
    mplsh(pi);
    mpsqrt(sx,sxi,x);                        Set Y_0 = 2^{1/4}.
    mpmov(y,sx);
    for (;;) {
        mpadd(z,sx,sxi);                     Set X_{i+1} = (X_i^{1/2} + X_i^{-1/2})/2.
        mplsh(z);
        mpsdv(x,z,2,ir);
        mpsqrt(sx,sxi,x);                    Form the temporary T = Y_i X_{i+1}^{1/2} + X_{i+1}^{-1/2}.
        mpmul(tt,y,sx);
        mplsh(tt);
```

```
            mpadd(tt,tt,sxi);
            mplsh(tt);
            x[0]++;                          Increment X_{i+1} and Y_i by 1.
            y[0]++;
            mpinv(ss,y);                     Set Y_{i+1} = T/(Y_i + 1).
            mpmul(y,tt,ss);
            mplsh(y);
            mpmul(tt,x,ss);                  Form temporary T = (X_{i+1} + 1)/(Y_i + 1).
            mplsh(tt);
            mpmul(ss,pi,tt);                 Set π_{i+1} = T π_i.
            mplsh(ss);
            mpmov(pi,ss);
            mm=tt[0]-1;                      If T = 1, then we have converged.
            for (j=1;j < n-1;j++)
                if (tt[j] != mm) break;
            if (j == n-1) {
                mp2dfr(pi,s);
                Convert to decimal for printing. NOTE: The conversion routine, for this demon-
                stration only, is a slow (∝ N^2) algorithm. Fast (∝ N ln N), more complicated,
                radix conversion algorithms do exist.
                s.erase(Int(2.408*np),s.length());
                return s;
            }
        }
    }
```

Figure 22.7.1 gives the result, computed with n = 1000. As an exercise, you might enjoy checking the first hundred digits of the figure against the first 12 terms of Ramanujan's celebrated identity [3]

$$\frac{1}{\pi} = \frac{\sqrt{8}}{9801} \sum_{n=0}^{\infty} \frac{(4n)!\,(1103 + 26390n)}{(n!\,396^n)^4} \tag{22.7.6}$$

using the above routines. You might also use the routines to verify that the number $2^{512} + 1$ is not a prime, but has factors 2,424,833 and 7,455,602,825,647,884,208,337,395,736,200,454,918,783,366,342,657 (which are in fact prime; the remaining prime factor being about 7.416×10^{98}) [4].

CITED REFERENCES AND FURTHER READING:

Borwein, J.M., and Borwein, P.B. 1987, *Pi and the AGM: A Study in Analytic Number Theory and Computational Complexity* (New York: Wiley).[1]

Knuth, D.E. 1997, *Seminumerical Algorithms*, 3rd ed., vol. 2 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), §4.3.[2]

Ramanujan, S. 1927, *Collected Papers of Srinivasa Ramanujan*, G.H. Hardy, P.V. Seshu Aiyar, and B.M. Wilson, eds. (Cambridge, UK: Cambridge University Press), pp. 23–39.[3]

Kolata, G. 1990, June 20, "Biggest Division a Giant Leap in Math," *The New York Times*.[4]

Kronsjö, L. 1987, *Algorithms: Their Complexity and Efficiency*, 2nd ed. (New York: Wiley).